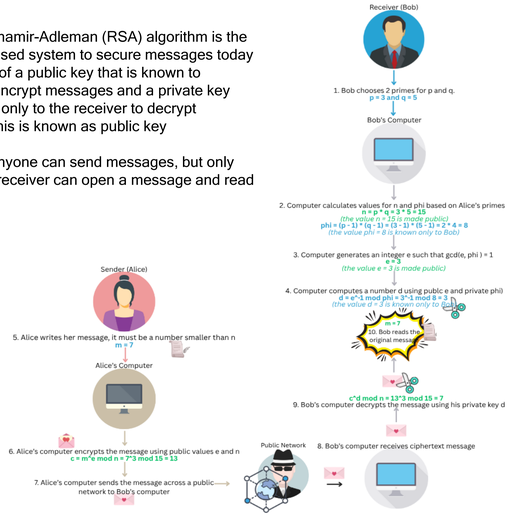
How Safe Are Your Online Transactions? A Cryptanalysis of RSA

Farwah Mirza, Mentor: Ellen Ziliak | Department of Mathematical and Computational Sciences | Benedictine University

The RSA Algorithm: Background Context

What is RSA?:

- The Rivest-Shamir-Adleman (RSA) algorithm is the most widely used system to secure messages today
- It makes use of a public key that is known to everyone to encrypt messages and a private key that is known only to the receiver to decrypt messages. This is known as public key cryptography.
- In this way, anyone can send messages, but only the intended receiver can open a message and read it



Beginnings of RSA:

- When it was first published in 1977, RSA was generally rejected due to its being one of the first public key cryptosystems
- The same year, the National Security Agency (NSA) proposed a federal Data Encryption Standard (DES) which implements a symmetric cryptosystem which was more popular at the time
- DES also ran much faster since it required a smaller key size than RSA
- In the end, a hybrid cryptosystem was made using the speed of DES combined with the security of a public-key system of RSA

Evolution of RSA: Key Size:

- The first official paper published by Rivest, Shamir, and Adleman recommended a key size of 663-bits (200 decimal digits) long
- When the industry standard came out with 1024-bits (309 decimal digits) as the standard key size for cryptosystems, RSA adjusted accordingly
- Now, the RSA algorithm uses key sizes within the 1024-bit to 2048-bit range
- Due to limited processing power as well as for research purposes, we used keys up to 64-bits (20 decimal digits) in our implementation of RSA

Mimicking RSA:

This is our implementation of the basic encryption-decryption piece of the RSA algorithm

```
import math
#Get 2 primes from sender (these are private values)
p = int(input("Enter a prime number p: "))
q = int(input("Enter a prime number q: "))

#Sender inputs message and all related computations are
m = int(input("Enter your message: "))
if m >= q:
    print("Your message must be less than q!") #Ensures
else:
    n = p * q #this value is public
    k = (p - 1)*(q - 1) #this value is private
    for e in range(2, k):
        if math.gcd(e, k) == 1:
            break

d = pow(e, -1, k) #this is the private key

#Ecryption
c = pow(m, e, n) #encoded ciphertext
print("This is the encoded message:", c)

#Decryption
decoded_message = pow(c, d, n)
print("This is the decoded message:", decoded_message)
```

Cryptanalysis

Cryptanalysis is the process of analyzing cryptosystems to uncover secret messages encrypted by the system

To test the security of our code, we implement known attacks on the RSA algorithm. These attacks effectively expose vulnerabilities in our code. Based on those vulnerabilities, we fix up our code to further secure our system

We've conducted 2 attacks so far:

- Brute Force Attack: Guessing Messages
- Brute Force Attack: Factorizing n
- Weiner's Attack

First Brute Force Attack: Guessing Messages

- The first attack we consider is a brute force attack. This attack generates every possible message and passes it through the encryption algorithm to see if it matches with the encrypted message sent by the sender.
- To see if our code is secure or not, we check how long it takes for the attack to find the original message. The longer it takes for the brute force attack to run, the more secure our code proves to be.

```
Enter a prime number p: 3
Enter a prime number q: 5
Enter your message: 7

This is the public value n: 15
This is the private value phi_N: 8
This is the public value e: 3
This is the private value d: 3

The sender encrypts the message: 13

The receiver can decrypt the encoded message with their private key: 7

Let's conduct a brute force attack...
The hacker finds the original message: 7

It took 0.005685304949951172 seconds to crack the code!
```

Prime Size:

- When using 16-bit size primes for p and q, we noticed the run time for the brute force attack was very small
- We increased the number of bits to 32 and noticed it took a lot longer for the system to uncover our message using brute force
- Seeing the direct relationship between bit-size and system security, we tried to further increase our primes to 64-bit
- At this point, the system took so long to break our code, we determined that at 64-bits, our message could not be uncovered with brute force

Generating Large Primes:

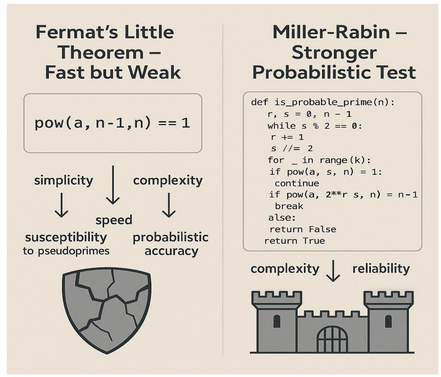
- We noticed that with primes we generated ourselves it was way too easy for our code to be hacked by brute force.
- One reason for this was that the prime numbers (p and q) that we were using to compute other values were too small. The real RSA algorithm uses up to 1064-bit primes which are hundreds of digits long
- To generate larger primes, we found a Python implementation of an existing algorithm called the Miller-Rabin Primality Test

Primality Check: Fermat's Little Theorem:

- Primality test based on the fact that if p is a prime number and x is relatively prime to p, then $x^{p-1} = 1 \bmod p$.
- In other words, if we want to test if a number p is prime, we can check it by plugging it into the FLT equation x^{p-1} and if we get a remainder of 1, the number is prime.
- If we get any other remainder or no remainder at all, the number is not prime

FLT Vulnerabilities:

- This test works well in general but has a vulnerability: Carmichael numbers. Carmichael numbers are numbers that pass as prime when plugged into the FLT equation but are not actually prime.
- For example, let's use x = 2 and p = 561 (a Carmichael number). Then $2^{561-1} \bmod 561 = 1$ but, 561 = 3 * 11 * 17 so it is clearly composite!
- To prevent Carmichael numbers from being passed as prime in our test, we combine Fermat's Little Theorem with a more secure and accurate probabilistic primality test known as Miller-Rabin



Miller- Rabin Primality Test

```
Set s to 0
Set r to the number minus 1

While r is even:
    Divide r by 2
    Increase s by 1

Repeat 10 times: #The more the following steps are repeated, the greater accuracy of the test
    Pick a random number 'a' between 2 and n-2
    Set x = a^r mod n #FLT equation modified with Miller-Rabin

If the number is 2 or 3:
    It is a prime (return True)

If the number is less than or equal to 1 OR the number is even:
    It is not a prime (return False)

Set s to 0
Set r to the number minus 1

While r is even:
    Divide r by 2
    Increase s by 1

Repeat 10 times: #The more the following steps are repeated, the greater accuracy of the test
    Pick a random number 'a' between 2 and n-2
    Set x = a^r mod n #FLT equation modified with Miller-Rabin

#Miller-Rabin extra tests for greater accuracy
    If x is not 1 and not n - 1:
        Repeat (s - 1) times:
            Set x = x^2 mod n

            If x becomes 1:
                It is not a prime (return False)

            If x becomes n - 1:
                Stop checking and go to the next round

        If the loop didn't stop early:
            It is not a prime (return False)

If it passed all tests:
    It is probably a prime (return True)
```

Miller-Rabin Primality Test: This is the first part of the Miller-Rabin code which modifies the FLT equation. Let's do an example to better understand this:

- Choose n = 11
- Then, s = 0 and r = 11 – 1 = 10
- Since 10 % 2 == 0, the while statement is true, and the loop is initiated
- 10 //= 2 --> 10/2 = 5, the floor of 5 is 5 --> so, r = 5
- s += 1 --> s = 0 and 0 + 1 = 1 --> so, s = 1
- Let's check if the loop statement is true:
- r = 5 --> 5 % 2 = 1 --> so, the while statement is false, and the loop terminates
- x = $a^r \bmod n$
- r = 5
- n = 11
- a = {2, 3, 4, 5, 6, 7, 8, 9, 10} --> a number chosen from this range
- Since we are doing 10 rounds, we will choose 10 integers for a (I'm only going to do one round in this example). According to FLT, if the Greatest Common Factor between a and n is 1, then x will be equivalent to 1, and n is prime.
- Choose a = 2
- x = $2^5 \bmod 11$
- = 32 mod 11 = 10

Let's continue with the next part of our example:

- We computed x = 10
- Now, the if statement asks 2 things:
- Is x = 1? --> No. --> Pass
- Does x = n – 1--> n = 11 and 11-1 = 10 --> Yes. --> Fail
- Since x = 10 and n – 1 = 10, the if statement is false and we skip the whole thing and go down to the else statement which requires us to return True.

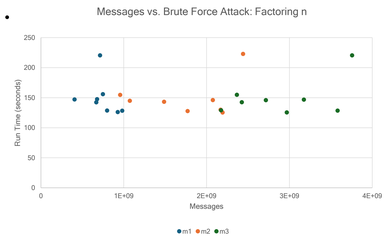
This for loop will run again and again for i number of iterations and if it returns true every time, the number will be considered prime.

Second Brute Force Attack: Factoring n:

- We implement a cleverer brute force attack on our updated encryption algorithm.
- This attack loops through all the factors of n and multiplies each of them together until they give us the correct values for p and q.

Data From the Attack:

- We wanted to test if the size of the user's message (m) had any effect on how fast the system could decrypt the message
- We used 3 messages of different size ranges: m1 ranges from 1 to $\frac{q}{3}$, m2 is within $\frac{q}{3}$ to $\frac{2q}{3}$, and m3 ranges from $\frac{2q}{3}$ to q.
- From the graph, we can see that our data is averaging at 150 seconds for the attack to uncover our message. Here we can see that no matter the size of the message, our run time stays consistent.
- This is interesting because we expect that larger messages take longer to crack. This is something we would like to further investigate in the future.



Small Private Exponent

Weiner's Theory:

- The Weiner's Attack is effective on all encryption algorithms that use a small decryption key (d) value
- If a d value is less than $(\frac{1}{3})n^{\frac{1}{4}}$ where n is the product of our primes, we can mathematically find the key value of phi(N) which can lead to finding d

RSA Derivation:

1. Start with the RSA equation for computing the private value d
d = $e^{-1} \bmod phi(N)$
2. By definition of inverse, this equation can be rewritten as
e x d = 1 mod phi(N)
3. Subtract k x phi(N) on both sides to get
(e x d) - (k x phi(n)) = 1
4. Divide both sides by d x phi(N), expand, and simplify to get
$\frac{e}{phi(N)} - \frac{k}{d} = \frac{1}{d*phi(N)}$
5. Since d and phi(N) are both very large numbers, e/phi(N) - (k/d) is equal to something very small. In other words, the difference between the two is very small, so they are really close.
According to Legendre's Theorem, we can assume that phi(N) is approximately equal to N since:
phi(N) = (p – 1)(q – 1) = (p x q) - (p + q) + 1 = N - (p + q) + 1
6. Since phi(N) is almost the same as N, we can replace phi(N) with N. Combine this with the fact that e/phi(N) is very close to k/d and we get the foundational equation of Weiner's Attack:
$\frac{e}{N} = \frac{k}{d}$
7. From here, Weiner's Attack expands e/N into a continued fraction. Out of all the convergents of the continued fraction, one of them will be k/d, our decryption key.

Weiner's Attack Example:

We're pretending to be a hacker, so from our perspective we only know the public key (e, n). Let's start with those values: e/N = 42667/64741

1. Let's expand this into a continued fraction using the Euclidean algorithm:
   - 42667 ÷ 64741 = 0 remainder 42667
   - 64741 ÷ 42667 = 1 remainder 22074
   - 42667 ÷ 22074 = 1 remainder 20593
   - 22074 ÷ 20593 = 1 remainder 1481
   - 20593 ÷ 1481 = 1 remainder 1340
   - 1481 ÷ 1340 = 1 remainder 141
   - 1340 ÷ 141 = 9 remainder 71
   - 141 ÷ 71 = 1 remainder 70
   - 71 ÷ 70 = 1 remainder 1
   - 70 ÷ 1 = 70 remainder 0

2. Now, we can calculate the convergents according to the following formula: hn = an* h(n − 1) + h(n−2) and kn = an k(n−1) +k(n−2)
   - H0 = 0 and k0 = 1
   - H1 = 1 * 0 + 1 = 1 and k1 = 1 * 1 = 1
   - H2 = 1 * 1 + 0 = 1 and k2 = 1 * 1 + 1 = 2
   - H3 = 1 * 1 + 1 = 2 and k3 = 1 * 2 + 1 = 2
   - H4 = 13 * 2 + 1 = 27 and k4 = 13 * 3 + 2 = 41
   - H5 = 1 * 27 + 2 = 29 and k5 = 1 * 41 + 3 = 44
   - H6 = 9 * 29 + 27 = 288 and k6 = 9 * 44 + 41 = 437
   - H7 = 1 * 288 + 29 = 317 and h7 = 1 * 437 + 44
   - H8 = 1 * 317 + 288 = 605 and k8 = 1 * 481 + 437 = 918
   - H9 = 70 * 605 + 317 = 42667 and k9 = 70 * 918 + 481 = 64741

So, our convergents are: [0/1, ½, 1/3, 2/3, 27/41, 29/44, 288/437, 317/481, 605/918, 42667/64741]

3. Next, we will run these convergents through Weiner's 3 tests. If a number passes all 3 tests, we can assume that it represents our private key k/d.

Test 1: Check if the equation phi(N) = ((e x d) - 1) / k returns a whole integer value.

- phi(n) = ((42667 x 1) - 1)/0 = undefined FAIL
- phi(n) = ((42667 x 2) - 1)/1 = 85333 PASS
- phi(n) = ((42667 x 3) - 1)/1 = 128000 PASS
- phi(n) = ((42667 x 3) - 1)/2 = 64000 PASS
- phi(n) = ((141 x 41) - 1)/27 = 214.074074 FAIL
- phi(n) = ((42667 x 44) - 1)/27 = 69531.3704 FAIL
- phi(n) = ((42667 x 437) - 1)/288 = 64741.2431 FAIL
- phi(n) = ((42667 x 481) - 1)/317 = 64740.776 FAIL
- phi(n) = ((42667 x 918) - 1)/605 = 64741 PASS
- phi(n) = ((42667 x 64741) - 1)/42267 = 65353.686 FAIL

Since p and q are whole integers, phi(N) should also be a whole integer

Test 2: Discard all convergents that fail at step 1 and check if the equation p + q = N − phi(N) + 1 returns a positive value.

- p + q = 64741 - 85333 + 1 = -20591 FAIL
- p + q = 64741 - 128000 + 1 = -63258 FAIL
- p + q = 64741 - 64000 + 1 = 742 PASS
- p + q = 64741 - 64741 + 1 = 1 PASS

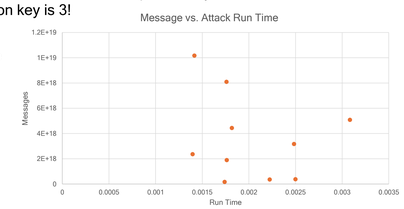
Since p and q are positive values, p + q should also be a positive value
Discard all convergents that fail in step 2. Find the discriminant of the quadratic equation x^2 - (p q) * x + N = 0 using the quadratic formula.

- X^2 - 742x + 64741 = 0
- Plug equation into the Quadratic Formula:
- X = [742 ±$\sqrt{742^2 - 4 * 1 * 64741}$] / 2 * 1
- X [742 ± 540]/2
- X = (742 + 540)/2 and x = (742 - 540)/2
- X = 641 and x = 101

So, we can conclude that our private key k/d = 2/3 which means our decryption key is 3!

Data on the Attack:

- We noticed that the message size doesn't have much of an influence over the run time of the attack
- We also notice that although the size of the public exponent e has some influence over the size of d, the effect is not significant enough to have an impact on the attack.



Future Research

- Investigate modifications to RSA that do not use modular arithmetic but perhaps provide more security to the risks associated with Quantum computing.

Works Cited

- Burger, E. B., & Starbird, M. (2013). Public-key cryptography. In *The heart of mathematics: An invitation to effective thinking* (4th ed., pp. 687-704). Wiley.
- Diffie, W., & Hellman, M. (1976). *New directions in cryptography* [PDF]. Stanford University. https://www.cs.virginia.edu/~evans/greatworks/diffie.pdf
- Rivest, R. L., Shamir, A., & Adleman, L. (1978). *A method for obtaining digital signatures and public-key cryptosystems*. Communications of the ACM, 21(2), 120-126. https://people.csail.mit.edu/rivest/Rsapaper.pdf
- Boneh, D. (n.d.). *Twenty years of attacks on the RSA cryptosystem* [PDF]. Stanford University. https://crypto.stanford.edu/~dabo/papers/RSA-survey.pdf
- Narayanan, A. (2014). *RSA: Mathematics and Attacks* [PDF]. MIT PRIMES. https://math.mit.edu/research/highschool/primes/materials/2014/conf/5-1-Narayanan.pdf
- Packet Mania. (2023, November 17). *RSA attack & defense #2 - low private exponent attack*. https://www.packetmania.net/en/2023/11/17/RSA-attack-defense-2/#low-private-exponent-attack
- Computerphile. (2017, March 23). *How RSA Works (Public Key Cryptography)* [Video]. YouTube. https://youtu.be/OpPrrndyYNU
- Microsoft Copilot. (n.d.). *Chat about RSA cryptography*. https://copilot.microsoft.com/chats/jG64gahZzSqQV5Gqw4SGX